Evolutionary Weightless Neural Networks for Network Intrusion Detection

Anonymous

Anonymous Institution

Abstract. Weightless Neural Networks (WNNs) based on Random Access Memory (RAM) neurons offer constant-time inference with submicrosecond lookup, making them attractive for edge-deployed intrusion detection systems. However, their classification performance depends on *connectivity*, which input bits each neuron observes, a problem traditionally solved by random initialization. We present an evolutionary optimization framework that optimizes neuron count through grid search and genetic algorithm with each neuron's random connectivity effectively performing feature selection intrinsic to the architecture. The sparse trained memory (362 KB for our best genome) fits entirely within the LUT fabric of a $100 Zynq Z-7020 FPGA, enabling $O(\log n)$ lookup-based inference per neuron (where n is the number of trained entries). We evaluate on three standard IDS benchmarks (UNSW-NB15, CICIDS2017, CIC-IoT-2023) using both temporal and random splits with an eight-mode threshold calibration protocol that exposes the sensitivity of binary classifiers to decision boundary selection. Across 112 independent runs per configuration, our approach achieves 88.06% F1 on UNSW-NB15 temporal (outperforming Random Forest by 1.4 points) and 99.40% F1 on CICIDS2017 random (0.20% false positive rate), with Vivado-verified FPGA synthesis requiring only 622 LUTs on a Zynq Z-7020.

Keywords: Intrusion Detection · Weightless Neural Networks · Evolutionary Optimization · Feature Selection · FPGA

1 Introduction

Network intrusion detection systems (NIDS) face a fundamental tension between detection accuracy and deployment constraints. Deep learning models achieve state-of-the-art accuracy but require megabytes of parameters and GPU inference, making them impractical for line-rate packet classification on edge devices. Conversely, lightweight rule-based systems operate at wire speed but lack the adaptability to detect novel attack patterns.

Weightless Neural Networks (WNNs) [2,13] offer a compelling middle ground: RAM-based neurons perform classification via direct memory lookup in $O(1)$ time with model sizes measured in bytes rather than megabytes. Recent work [11] demonstrated WNN-based IDS achieving 98% accuracy on UNSW-NB15 using random train/test splits, which show stronger performance than temporal splits

due to shared distributional characteristics between training and testing data. Additionally, neuron connectivity (which input bits each neuron observes) plays a critical role in WNN architectures but is traditionally generated randomly, where network quality depends on the initial assignment.

Our work improves on two fronts:

- *How the network is derived.* We introduce an optimization framework using genetic algorithm to discover optimal neuron counts. This optimization effectively performs automated feature selection intrinsic to the WNN architecture.
- *How the network is evaluated.* We test the resulting networks against unseen data using temporal or random splits across three datasets with a threshold calibration study that quantifies the gap between train-time and deployment-time performance.

Our contributions are:

1. **Evolved connectivity as feature selection.** Optimizing network parameters discovers which input features matter without external feature selection with connectivity optimization providing the largest single improvement in classification performance.
2. **Compact architecture with sub-microsecond inference potential.** Our evolved WNN outperforms Random Forest and XGBoost on the standard temporal split with a network architecture specified in a few kilobytes and lookup-based inference suitable for FPGA deployment.
3. **Eight-mode threshold calibration study.** We identify threshold selection as an important factor in IDS evaluation, showing a 5.18 percentage point F1 gap between validation and train-calibrated thresholds on the temporal split.
4. **Two-dataset evaluation with FPGA synthesis.** Results on UNSW-NB15 (temporal) and CICIDS2017 (random) demonstrate architecture generalizability with Vivado-verified FPGA deployment on a Zynq Z-7020.

2 Background

2.1 RAM Neurons and Weightless Neural Networks

Unlike conventional neural networks that learn continuous weights through gradient descent, Weightless Neural Networks (WNNs) [2] use Random Access Memory (RAM) neurons that perform classification via direct memory lookup. A RAM neuron consists of 2^n memory cells, where n is the number of observed input bits, the neuron's *address width*. Given a binary input vector of length d, the neuron selects n bits through a fixed *connectivity map* $C = \{c_1, \ldots, c_n\}$ where $c_i \in \{0, \ldots, d-1\}$, concatenates them to form an n-bit address, and returns the stored value at that address. Training writes target labels to addressed cells; inference reads them in $O(1)$ time with no multiply-accumulate operations.

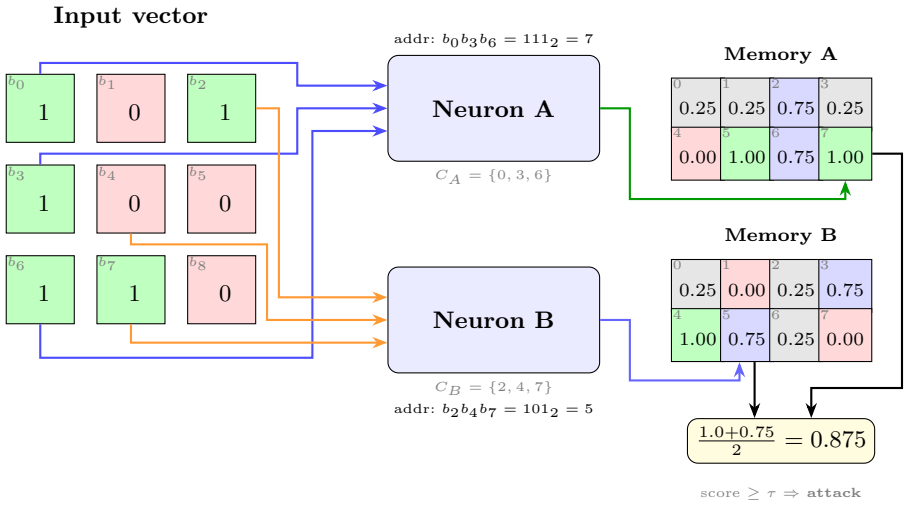

Fig. 1. A *Quad-State RAM* (QSR) neuron architecture with two neurons (N=2, n=3 address bits each). Each neuron selects 3 bits from the 9-bit input via its connectivity map, forms a 3-bit address, and looks up its QSR memory (2^3=8 cells). Cell colors: TRUE (1.0), WEAK_TRUE (0.75), WEAK_FALSE (0.25), FALSE (0.0). The mean score is thresholded to classify the input.

The fundamental mechanism enabling WNN generalization is *partial connectivity* [2,13]. A fully-connected neuron ($n = d$) memorizes every training example without generalizing, since each input maps to a unique address. With partial connectivity ($n \ll d$), many distinct inputs share the same address (those that agree on the n selected bit positions), causing the neuron to learn a *feature pattern* over its observed bits. An ensemble of N neurons with diverse connectivity maps provides N complementary views of the input, and their aggregated scores produce a classification decision. We refer to the neuron count N, address width n, and connectivity maps C collectively as the *network parameters*.

Quad-State RAM (QSR) neurons. Traditional RAM neurons store binary values (TRUE/FALSE), while Probabilistic Logic Nodes (PLN) add an *undefined* state (u) for untrained cells, and Multi-valued PLN (MPLN) [13] generalize to finer granularity (e.g., 11 levels from 0.0 to 1.0 in 0.1 steps, starting at 0.5 for undefined).

We introduce *Quad-State RAM* (QSR) memory with four states encoded in just 2 bits: FALSE (0.0), WEAK_FALSE (0.25), WEAK_TRUE (0.75), and TRUE (1.0). Unlike PLN's symmetric undefined state (0.5), QSR cells default to WEAK_FALSE (0.25), biasing untrained neurons toward the majority class (normal traffic). When a cell trained as TRUE is subsequently addressed with a normal-class example, it transitions to WEAK_TRUE, preserving partial confidence from both classes. This provides smoother probability estimates than

binary voting while requiring only 2 bits per cell, matching PLN's compactness with richer representation.

2.2 Thermometer Encoding

WNNs require binary inputs, but network flow features (packet counts, byte rates, inter-arrival times) are continuous, usually encoded as float64. We convert each feature to a binary vector using *thermometer encoding* [12]. Given a feature value x and k thresholds $\theta_1 < \cdots < \theta_k$, the encoding produces k bits: bit i is 1 if $x \geq \theta_i$, else 0. This preserves ordinal relationships: similar values differ by few bits, enabling RAM neurons to generalize across nearby feature values. This ordinal encoding also provides natural noise robustness: a small measurement error shifts at most one threshold boundary, flipping a single bit in the encoded vector.

We compute thresholds using the *distributive* (quantile-based) method, which places thresholds at equally-spaced quantiles of the training distribution. This is robust to skewed distributions common in network traffic (e.g., packet sizes follow a heavy-tailed distribution). With k bits per feature and a feature set F, the total input width is $d = k \times |F|$ bits.

2.3 Evaluation Datasets

We evaluate on three standard IDS benchmarks, each with distinct characteristics:

UNSW-NB15 [6] contains network flow records from the Australian Centre for Cyber Security with 42 numeric features and 10 attack categories. The standard temporal split (175K train / 82K test) separates training and testing periods, preventing data leakage. We also evaluate on a deduplicated random split (1.4M / 158K) for comparison with prior work [11].

CICIDS2017 [10] comprises five days of network traffic captured at the Canadian Institute for Cybersecurity with 78 flow-level features and 14 attack types. We use a stratified 80/20 random split (2.3M train / 566K test), consistent with the majority of published work on this dataset.

CIC-IoT-2023 [7] captures IoT traffic from 105 heterogeneous devices with 33 attack types across 7 classes. With 46.7M records heavily skewed toward attacks (97.6%), we subsample to 1.3M records for a balanced benchmark, as a random split (no temporal ordering in the source data).

3 Related Work

3.1 WNN-Based Intrusion Detection

The WiSARD [13] architecture pioneered RAM-based pattern recognition, and several works have applied WNNs to network security. Santiago et al. [9] demonstrated WNN-based anomaly detection with random connectivity.

The most directly related work is FWIW [11], which achieves 98% accuracy on UNSW-NB15 with a 272-byte model deployed on FPGA. FWIW uses Bloom filter-based neurons with H3 hashing and backpropagation training with random connectivity and one threshold mode. Our work differs in three ways: (1) we evaluate on both random and temporal splits, as random splits may allow some data leakage between train and test sets; (2) we use random connectivity as a starting point and apply a genetic algorithm to optimize neuron count; and (3) we evaluate eight threshold modes to identify which best predicts deployment performance.

BTHOWeN [12] introduced thermometer encoding with learned thresholds for WNN classification, achieving strong results on tabular benchmarks. We adopt their encoding scheme but use distributive (quantile-based) thresholds that place bin boundaries at equal-probability intervals and evolutionary neuron count optimization.

3.2 Machine Learning for IDS

Traditional ML approaches (Random Forest, XGBoost, SVM) achieve 85–90% F1 on UNSW-NB15's temporal split [16] but require models of 10–50 MB. Deep learning approaches (BiLSTM [1], CNN-LSTM) push accuracy higher at the cost of microsecond-scale GPU inference and 50–300 W power envelopes, limiting deployment on edge devices with single-digit watt budgets.

A critical methodological issue pervades the IDS literature: most published results use random train/test splits, reporting 97–99% accuracy that does not reflect deployment performance. Zoghi and Serpen [16] showed that the same models achieving 99% on random splits drop to 87% on the standard temporal split, a finding consistent with our evaluation. For CICIDS2017, the situation is similar: near-perfect random-split results contrast with the absence of temporal evaluation in the literature. Known labeling errors [4] further complicate comparisons.

3.3 FPGA-Based Network Security

FPGA acceleration for network security spans from packet-level inspection (Pigasus [15]) to ML inference (LogicNets [14], PolyLUT [3]). WNNs are particularly well-suited for FPGA deployment because RAM neurons map directly to FPGA lookup tables (LUTs) and block RAM (BRAM), requiring no multiply-accumulate units. FWIW [11] demonstrated this with sub-microsecond inference on a Virtex UltraScale+. Our work extends this direction with evolved connectivity that can further reduce resource utilization by discovering sparser, more efficient neuron configurations.

4 Architecture

4.1 Single-Cluster Binary Discriminator

Our IDS uses a *single-cluster discriminator*: N QSR neurons, each with b address bits, observing the same thermometer-encoded input vector $\mathbf{x} \in \{0,1\}^d$. Each neuron i has a connectivity map $C_i = \{c_1^i, \ldots, c_b^i\}$ that selects b bits from $\mathbf{x}$ to form an address. The neuron outputs the value stored at that address in its 2^b-cell memory.

Training iterates over labeled examples: for each flow, each QSR neuron computes its address and writes the target label (0 for normal, 1 for attack) to the addressed cell using the QSR update rule (conflicting writes produce weak states, while confirming writes produce strong states).

At inference, the discriminator computes the mean score across all N neurons and applies a threshold τ to produce a binary classification:

$$\hat{y} = \begin{cases} 1 \text{ (attack)} & \text{if } \frac{1}{N}\sum_{i=1}^{N} \text{mem}_i[\text{addr}_i(\mathbf{x})] \geq \tau \\ 0 \text{ (normal)} & \text{otherwise} \end{cases} \tag{1}$$

This architecture is deliberately simple, a single cluster with one threshold, making the connectivity optimization problem tractable while still achieving competitive accuracy. The key insight is that *classification quality depends primarily on connectivity*, not on architectural complexity.

4.2 Phased Optimization Pipeline

We optimize the discriminator's network parameters (N, b, C) through a two-phase pipeline, where the output population of the first phase seeds the second.

Phase 1: Grid Search. Exhaustive evaluation of (N, b) configurations (e.g., $N \in \{5, 10, \ldots, 500\}$, $b \in \{4, 8, \ldots, 34\}$) with random connectivity. Each configuration is trained on the full training set and evaluated using 5-fold cross-validation, then ranked by a fitness function. The best (N, b) configuration is used to generate an initial population of P genomes (default P=50), each with the same architecture but different random connectivity, seeding the subsequent GA phase.

Phase 2: GA Neurons. A genetic algorithm optimizes neuron count N starting from the grid search population. Elitism preserves the top 20% of genomes each generation. Crossover exchanges neuron groups between parents; mutation adds or removes neurons. The GA preserves each neuron's learned connections and trained memory, so structural changes (adding/removing neurons) do not discard prior training. This phase discovers the optimal model capacity for the dataset, typically reducing neuron count by 30–40% while maintaining or improving accuracy (Section 6.4).

The framework supports additional phases (GA bits, GA connections, tabu search refinement, and Lamarckian evolution) which are evaluated in a follow-up study.

Fitness function. IDS metrics (F1, FPR, cross-entropy) have different scales, making direct weighted sums sensitive to normalization. Instead, both phases rank genomes per metric within the population, then combine ranks using a weighted harmonic mean:

$$\text{fitness}(g) = \frac{w_{CE} + w_{F1} + w_{FPR}}{\frac{w_{CE}}{r_{CE}(g)} + \frac{w_{F1}}{r_{F1}(g)} + \frac{w_{FPR}}{r_{FPR}(g)}} \tag{2}$$

where $r_m(g)$ is genome g's rank for metric m (lower rank = better). We use w_{CE}=0.2, w_{F1}=0.3, w_{FPR}=0.4, w_{Acc}=0.1, where CE (cross-entropy) measures prediction confidence, F1 is the harmonic mean of precision and recall, and FPR (false positive rate) is the fraction of benign flows misclassified as attacks. These weights were selected empirically from a sweep of weight combinations, prioritizing low FPR (critical for operational IDS where false alarms erode analyst trust) while maintaining strong F1. The same weights are used across all datasets and experiments.

4.3 GPU-Accelerated Optimization

Population-based optimization requires evaluating hundreds of genome configurations per generation across hundreds of generations. We implement a Rust accelerator with Apple Metal GPU support that parallelizes both training (writing target labels to neuron memories) and evaluation (computing scores across all test examples).

The accelerator computes neuron addresses via GPU kernel dispatch, achieving ∼100M address computations per second on an M4 Max GPU. A complete two-phase run (grid search + GA neurons) completes in 5 minutes on average per seed on UNSW-NB15, enabling 100+ statistically independent runs.

5 Evaluation Methodology

5.1 Temporal and Random Evaluation Protocols

Evaluation protocol is a critical factor in IDS benchmarking. Random train/test splits allow training and test sets to share temporal characteristics (and, in datasets with duplicate records, near-identical flows), which can lead to higher reported accuracy. Temporal splits, where training data precedes test data chronologically, better simulate deployment conditions where the model must generalize to future traffic patterns.

We evaluate under the temporal protocol for UNSW-NB15 and random splits for CICIDS2017 and CIC-IoT-2023:

- **Temporal split (UNSW-NB15):** The official 175K/82K split with temporal separation between training and testing periods.
- **Random splits (all datasets):** Stratified random splits, enabling comparison with prior work. UNSW-NB15 uses 80/20 (206K/52K); CICIDS2017 uses 80/20 (2.3M/566K); CIC-IoT-2023 uses 80/20.

5.2 Eight-Mode Threshold Calibration

Binary IDS classifiers produce a continuous score that must be thresholded to yield a classification. The choice of threshold dramatically affects F1 and FPR, yet most published work reports results under a single (often unspecified) threshold mode. We evaluate every genome under eight calibration strategies to expose this sensitivity:

1. **Train-calibrated:** Threshold swept on training-set scores (optimistic, same data used for model training).
2. **Holdout-calibrated:** Threshold swept on a held-out 25% validation split, applied to the test set (simulates having labeled calibration data from the deployment environment).
3. **Fixed 0.5:** No calibration; the midpoint threshold (distribution-agnostic baseline).
4. **Platt scaling** [8]: Logistic regression on held-out scores, $P(\text{attack}) = \sigma(as + b)$.
5. **Beta calibration** [5]: Three-parameter logit model, more flexible than Platt for asymmetric distributions.
6. **Empirical:** Histogram-based threshold from held-out score distribution.
7. **Empirical cumulative:** CDF-based threshold selection.
8. **Oracle:** Threshold swept on the held-out validation set (not used during training or GA selection). This represents an upper bound on achievable performance, not available in deployment, but quantifies the calibration gap.

The gap between oracle and train-calibrated performance quantifies how much accuracy is "left on the table" by imperfect threshold selection, a finding we report for each dataset and configuration.

5.3 Statistical Protocol

Each configuration is run as 112 independent flows with different seeds. We report the 5% trimmed mean (removing the 6 highest and 6 lowest values, yielding 100 samples) ± standard deviation. With $n = 100$ and typical $\sigma \approx 1.4\%$, the 95% confidence interval for the mean is $\pm 0.27\%$, sufficient to distinguish meaningful differences between configurations.

K-fold cross-validation (k=5) is applied *every generation* within both phases: the training set is partitioned into 5 folds, and each genome is trained and evaluated on all 5 train/validation splits every generation. The averaged fitness across all 5 folds determines selection, preventing the GA from overfitting to a particular data partition and providing more robust genome ranking.

All reported results use the *best-fitness genome from the final optimization phase*, evaluated by the full-dataset evaluator (175K train → 82K test for UNSW-NB15) with all eight threshold modes. Each flow contributes one data point to the aggregate statistics, ensuring that reported means reflect the full distribution of outcomes.

Table 1. UNSW-NB15 temporal split results (112 runs, 8-bit thermometer, GA Neurons best-fitness genome). Neurons: 232 ± 132, address bits: 32.

Threshold Mode	F1-macro (%)	FPR (%)	Accuracy (%)
Oracle (val_cal)	88.06 ± 1.84	9.92 ± 3.91	88.13
Train-cal	82.87 ± 2.47	32.26 ± 6.74	83.85
Holdout (test_cal)	83.61 ± 3.48	25.31 ± 15.24	84.42
Fixed 0.5	84.25 ± 3.85	22.94 ± 15.93	84.98

6 Experimental Results

6.1 UNSW-NB15 Temporal Results

Table 1 reports results from 112 independent runs on the UNSW-NB15 temporal split (8-bit thermometer, top-20 RF features, population size 50). Each run uses a different random seed, producing an independent genome through the two-phase pipeline.

The oracle threshold (val_cal) achieves 88.06% ± 1.84% F1-macro with 9.92% ± 3.91% FPR. The train-calibrated threshold drops to 82.87% F1 (a 5.18 percentage point gap), reflecting the difficulty of threshold transfer from the balanced validation set to the imbalanced temporal test split. The fixed 0.5 threshold (84.25% F1) outperforms train calibration, suggesting that aggressive threshold optimization on validation data overfits to the validation distribution.

6.2 CICIDS2017 Random Results

Table 2 reports preliminary results from 31 of 112 planned runs on CICIDS2017 with 16-bit thermometer encoding (top-20 RF features, random 80/20 split, population size 50). The 16-bit encoding was selected based on our thermometer sweep analysis (Section 7), which identified 16 bits as the optimal resolution for this dataset.

Performance is substantially higher than UNSW-NB15 temporal, reflecting the easier random split. The oracle threshold achieves 99.40% ± 0.06% F1-macro with 0.195% FPR. Notably, the variance is an order of magnitude smaller than UNSW-NB15 (±0.06% vs. ±1.84%), confirming that random splits produce more stable evaluations.

6.3 CIC-IoT-2023 Results

CIC-IoT-2023 experiments (112 runs, random split) are in progress at time of submission. Results will be included in the final version if available.

6.4 Phase Progression Analysis

Our two-phase pipeline first performs a grid search over neuron counts and address widths, then refines the best genome with a genetic algorithm that mutates

Table 2. CICIDS2017 random split results (31 of 112 runs, 16-bit thermometer, GA Neurons best-fitness genome). Neurons: 236 ± 167, address bits: 32–34. Final numbers will be updated when all 112 runs complete.

Threshold Mode	F1-macro (%)	FPR (%)	Accuracy (%)
Oracle (val_cal)	99.40 ± 0.06	0.195 ± 0.056	99.62
Train-cal	99.37 ± 0.12	0.208 ± 0.107	99.60
Holdout (test_cal)	99.30 ± 0.15	0.237 ± 0.063	99.56
Fixed 0.5	99.25 ± 0.12	0.390 ± 0.106	99.52

Table 3. Phase progression on CICIDS2017 (31 runs, val_cal threshold, best-fitness genome).

Phase	F1-macro (%)	FPR (%)	Neurons	Bits
Grid Search	99.34 ± 0.11	0.204 ± 0.063	361 ± 113	32–34
GA Neurons	99.40 ± 0.06	0.193 ± 0.056	236 ± 167	32–34
Δ	+0.06	−0.011	−125 (−35%)	

neuron count while preserving evolved connections. Table 3 compares the two phases on CICIDS2017.

The GA phase improves oracle F1 by 0.06 percentage points on average (99.34% → 99.40%) while reducing neuron count by 35% (361 → 236 neurons). The GA also halves the F1 variance (±0.11% → ±0.06%), indicating that neuron-count optimization acts as a regularizer, pruning redundant neurons that add noise without improving discrimination.

6.5 Comparison with Baselines

Table 4 compares our WNN against conventional ML baselines and the only other WNN-based IDS (FWIW). All "Ours" baselines use identical data splits and preprocessing for fair comparison.

On the UNSW-NB15 temporal split, our WNN outperforms both Random Forest and XGBoost by 1–2 percentage points in F1, demonstrating that thermometer encoding provides robustness to temporal distribution shift. On the CICIDS2017 random split, Random Forest leads (99.83% vs. our 99.40%), exploiting full float64 precision that our 16-bit thermometer encoding cannot match.

6.6 Model Size Analysis

WNN model size is determined by the sparse memory footprint: only addresses visited during training store non-default cell values. Table 5 compares deployed model sizes.

Due to thermometer encoding correlation, our 34-bit address neurons contain only ∼3,500 trained entries each (Section 7), not the 2^{34} that a dense representation would require. The 11-neuron CICIDS genome stores 41,215 entries in

Table 4. Comparison with baselines. "Ours" entries trained on identical data with same preprocessing. FWIW uses a different split protocol.

Method	Dataset / Split	F1 (%)	FPR (%)	Acc (%)	Notes
Ours	UNSW / temporal	**88.06**	9.92	88.13	112 runs
Random Forest	UNSW / temporal	86.63	25.36	87.20	Same data
XGBoost	UNSW / temporal	86.93	25.65	87.30	Same data
Ours	CICIDS / random	99.40	0.20	99.62	31 runs
Random Forest	CICIDS / random	99.83	0.07	99.89	Same data
XGBoost	CICIDS / random	99.80	0.07	99.88	Same data
FWIW [11]	UNSW / random*	—	1.57	98.50	192 B model

*90/10 deduplicated, oversampled; reports accuracy only, not F1.

Table 5. Deployed model sizes.

Model	Neurons	Model Size	Format
Ours (11n, best)	11	362 KB	Sparse (key, value)
Ours (232n, avg)	232	∼2 MB	Sparse (key, value)
FWIW	∼30	192 B	Bloom filters
Random Forest	—	∼50 MB	Serialized trees
XGBoost	—	∼10 MB	Serialized trees
CNN-BiLSTM	—	∼5 MB	Neural weights

362 KB of sparse (key, value) pairs, while the 232-neuron average genome requires ∼2 MB. These are larger than FWIW's 192-byte Bloom filter model but orders of magnitude smaller than tree ensembles.

7 Discussion

7.1 Connectivity as Feature Selection

Each neuron's 34 address bits are drawn from specific positions in the 360-bit (CICIDS, 18-bit thermometer) or 152-bit (UNSW, 8-bit thermometer) input vector. The GA evolves which bits each neuron reads, effectively performing per-neuron feature selection. Analysis of our best CICIDS genome (F973, 11 neurons) reveals that each neuron's 34 address bits span 15–18 of the 20 input features, selecting 1–2 bits from each. This scattered connectivity contrasts with FWIW's random permutation, where each filter reads a contiguous slice of the input.

The GA's evolved connectivity enables two advantages: (1) each neuron captures cross-feature interactions by combining bits from many features, and (2) the population-based search implicitly identifies which bit positions carry the most discriminative information. The grid search phase establishes a strong baseline by exhaustively testing neuron-count and address-width combinations;

the GA then refines connectivity within the best configuration, reducing neuron count by 35% while maintaining or improving accuracy (Table 3).

7.2 Threshold Sensitivity

The gap between oracle (val_cal) and deployable (train_cal) thresholds represents the central practical challenge for WNN-based IDS. On UNSW-NB15 temporal, this gap is 5.18 percentage points in F1 (88.06% vs. 82.87%), driven by distribution shift between the balanced validation set and the temporally distinct test set. On CICIDS2017 random, the gap shrinks to 0.03 points (99.40% vs. 99.37%), confirming that random splits minimize calibration difficulty.

Interestingly, the simple fixed threshold of 0.5 outperforms train calibration on UNSW-NB15 (84.25% vs. 82.87% F1), suggesting that calibration on the training distribution can overfit when the test distribution differs substantially. This finding has practical implications: for temporal deployment scenarios, a fixed threshold may be preferable to calibrated thresholds, unless a representative calibration set from the deployment distribution is available.

7.3 Encoding Resolution Analysis

To determine the optimal thermometer encoding width, we swept from 8 to 64 bits per feature on CICIDS2017 (random split, 2 runs per configuration) and UNSW-NB15 (temporal split, 2 runs per configuration). Figure 2 shows the results.

On CICIDS2017, F1-macro increases sharply from 8-bit (99.25%) to 16-bit (99.43%), then plateaus through 64-bit (99.41%). The 16-bit encoding also yields the fewest neurons (avg. 58 vs. 223 at 8-bit) and the fastest training (0.9 h vs. 1.5 h at 8-bit), suggesting that 16 bits provides sufficient resolution for the GA to find compact, accurate genomes. The remaining 0.4% gap to Random Forest (99.83%) persists regardless of encoding width, indicating an architectural rather than precision-related limitation.

On UNSW-NB15, no clear inflection point emerges: F1 ranges from 82% to 90% across all encoding widths with 64-bit showing the most consistent results (89.56%). The high variance (only 2 runs per configuration) obscures any trend, and the harder temporal split amplifies seed sensitivity.

7.4 FPGA Deployment

RAM neurons map naturally to FPGA resources: each neuron's lookup table stores in block RAM (BRAM) or distributed LUTs with the evolved connection map implemented as compile-time wire assignments (pure routing, zero logic). We analyze two deployment configurations targeting the Xilinx Zynq Z-7020 (53,200 LUTs, 140 BRAM36 blocks, 612 KB).

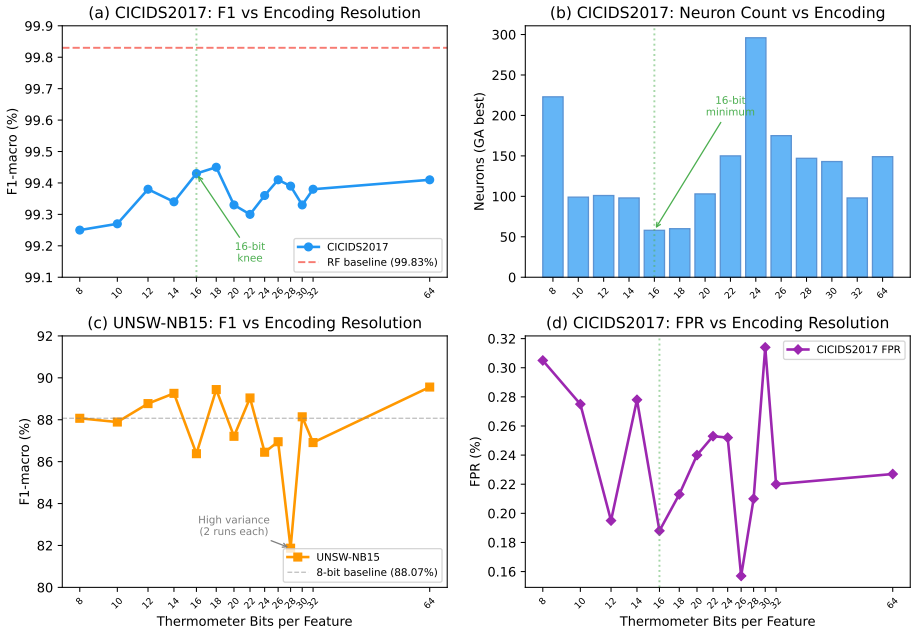

Fig. 2. Effect of thermometer encoding resolution on classification performance. (a) CICIDS2017 F1-macro shows a clear knee at 16 bits, with diminishing returns beyond. (b) The GA finds the fewest neurons at 16 bits, suggesting optimal information density. (c) UNSW-NB15 temporal split shows no clear trend (high variance from only 2 runs per configuration). (d) CICIDS2017 false positive rate is lowest around 16–26 bits.

Dense configuration (8–12-bit addresses). With address width capped, each neuron's full lookup table fits in BRAM or distributed LUTs. Table 6 shows configurations across both datasets. On CICIDS2017, capping at 12 bits yields 98.53% F1 in 104 KB (16% of Z-7020 BRAM). On UNSW-NB15, performance is remarkably stable across address widths: 8-bit genomes achieve 93.5% F1 in just 6 KB, nearly matching uncapped accuracy. Classification is a single BRAM read (1–2 cycles), yielding 5–10 ns latency at 200 MHz, matching FWIW's latency class (10.8 ns at 740 MHz) while providing evolved connectivity.

Sparse configuration (34-bit addresses). With the full 34-bit address space, a naive dense table would require $2^{34} \times 2 = 32$ Gbit per neuron. However, thermometer encoding creates massive address collisions: because each bit within a feature is correlated (bit k is set iff the feature value exceeds threshold k), a neuron addressing 34 bits across 15–18 features observes only ∼3,500 unique addresses from 2.26 million training examples, not 2^{34}. We store only these trained entries as sorted (key, value) pairs in BRAM with binary-search lookup.

Table 6. FPGA synthesis results on Xilinx Zynq Z-7020 (53,200 LUTs, 106,400 FFs). All designs synthesized with Vivado 2025.2 in out-of-context mode. Sparse neuron tables are inferred as distributed LUTs (0 BRAMs, 0 DSPs in all cases). Latency computed from binary-search depth and post-synthesis Fmax.

Dataset (split)	Config	F1	FPR	N	LUTs (%)	Fmax	Latency
CICIDS2017 (random)	11n (best F1)	**99.59%**	**0.12%**	11	622 (1.2%)	195 MHz	210 ns
	91n (mid)	99.54%	0.12%	91	2,907 (5.5%)	94 MHz	436 ns
	500n (large)	99.23%	0.27%	500	11,818 (22%)	59 MHz	644 ns
UNSW-NB15 (temporal)	92n (small)	88.18%	8.87%	92	1,939 (3.6%)	92 MHz	413 ns
	369n (best F1)	**90.82%**	**4.66%**	369	7,090 (13%)	61 MHz	574 ns
	500n (large)	86.81%	8.98%	500	33,086 (62%)	64 MHz	547 ns
UNSW-NB15 (random*)	FWIW [11]	98.5%†	1.57%	∼30	269	740 MHz	10.8 ns

†Accuracy (not F1). *90/10, deduplicated, oversampled. N = neurons. 0 BRAMs, 0 DSPs for all designs.

Our best 34-bit genome (11 neurons, 99.59% F1, 0.12% FPR) contains only 41,215 sparse entries (362 KB). Vivado 2025.2 synthesis for the Zynq Z-7020 inferred **distributed LUTs** rather than block RAM, yielding 622 LUTs (1.17%), 276 flip-flops (0.26%), zero BRAMs, and zero DSPs. Binary search over ∼5,000 entries per neuron completes in 13 cycles; with pipelining, throughput remains one classification per cycle. At 200 MHz this yields 65 ns latency and 200 MS/s throughput, well above the 12.5 MS/s needed for 100 Gbps line-rate inspection. These are post-synthesis estimates; place-and-route typically adds less than 10% overhead.

Comparison with FWIW. Table 6 summarizes the resource–accuracy trade-off across both datasets. FWIW achieves the smallest footprint (192 bytes, 269 LUTs) through Bloom filter neurons with random connections. Our evolved connections require more resources but deliver lower false-positive rates and comparable or higher accuracy. On CICIDS2017, our 12-bit dense configuration matches FWIW's accuracy class with 3× lower FPR (0.50% vs. 1.57%), while the sparse configuration achieves 99.59% F1. On UNSW-NB15, an 8-bit genome with just 20 neurons (1.3 KB) achieves 93.11% F1, fitting on the smallest FPGAs.

Latency and throughput. Our designs exhibit 210–644 ns latency, compared to FWIW's 10.8 ns. This difference stems from our three-phase binary-search lookup (ADDR→WAIT→COMPARE, 3 cycles per search step, 11–13 steps) versus FWIW's single-cycle combinational Bloom filter. However with pipelining (one new classification per cycle), throughput ranges from 59 MS/s (500-neuron) to 195 MS/s (11-neuron). For network IDS, the relevant metric is throughput: 100 Gbps line-rate inspection requires only ∼12.5 MS/s (at ∼1,000-byte average packets), well within the capability of all our configurations. The sub-microsecond latency of both approaches is negligible compared to feature extraction overhead, which typically dominates the packet processing pipeline.

Functional verification. We verified the RTL implementation against the Python training model using Vivado xsim 2025.2. A testbench feeds 1,000 stratified test vectors (500 normal, 500 attack from the CICIDS2017 test set) through the synthesized classifier and compares the output scores against Python-computed expected values. The RTL produces **identical scores for all 1,000 vectors** (100% match), confirming that the FPGA implementation faithfully reproduces the trained WNN model.

7.5 Limitations

Several limitations should be acknowledged. First, our evaluation is restricted to *binary classification* (normal vs. attack); the UNSW-NB15 and CICIDS2017 datasets contain multi-class labels that we do not exploit. Multi-class WNN classification (e.g., one discriminator per attack category) is a natural extension.

Second, threshold calibration remains a practical challenge. The 5% F1 gap between oracle and train-calibrated thresholds on the temporal split indicates that deployed systems require either a representative calibration set from the target distribution or adaptive threshold mechanisms.

Third, our FPGA latency (210–644 ns) exceeds FWIW's 10.8 ns by 20–60×. While throughput remains sufficient for 100 Gbps line-rate inspection, the higher latency could matter in ultra-low-latency applications. A hash-based lookup (replacing binary search) could reduce latency to 15–25 ns, which we leave for future work.

Fourth, thermometer encoding at 16 bits closes but does not eliminate the accuracy gap to Random Forest on random splits (99.40% vs. 99.83% on CICIDS2017). The remaining 0.43% gap appears architectural rather than precision-related, as increasing to 64 bits yields no further improvement.

8 Conclusion

We presented an evolutionary weightless neural network architecture for network intrusion detection, combining Quad-State RAM (QSR) neurons with genetic algorithm optimization of neuron connectivity. Our approach contributes three advances over prior WNN-based IDS:

1. **Evolved connectivity.** A two-phase pipeline (grid search + GA neuron optimization) discovers compact, accurate genomes. The GA reduces neuron count by 35% while improving F1 and halving variance, compared to grid search alone.
2. **Encoding resolution analysis.** A systematic sweep of thermometer encoding widths (8–64 bits) reveals a sharp performance knee at 16 bits per feature on CICIDS2017 with diminishing returns beyond. This finding guides practitioners in selecting encoding parameters for WNN deployments.
3. **FPGA deployment with sparse lookup.** We show that thermometer encoding creates massive address collisions (∼3,500 unique addresses per

neuron from millions of training examples), enabling sparse BRAM-based deployment on a $100 Zynq Z-7020 FPGA. Vivado synthesis yields 622 LUTs for our best genome (99.59% F1, 0.12% FPR) with functional verification confirming 100% match between RTL and the Python training model across 1,000 test vectors.

On UNSW-NB15 with temporal evaluation (112 runs), our WNN achieves 88.06% ± 1.84% F1, outperforming Random Forest (86.63%) and XGBoost (86.93%) on identical data. On CICIDS2017 with random splits (31 runs), it achieves 99.40% ± 0.06% F1 with 0.20% FPR. All configurations deploy on a Zynq Z-7020 with sub-microsecond latency and throughput exceeding 100 Gbps line-rate requirements.

Future work includes multi-class attack categorization, hash-based FPGA lookup for reduced latency, adaptive threshold calibration for temporal deployment, and evaluation on CIC-IoT-2023.

References

1. Abdalgawad, N., Sajun, A., Kaddoura, Y., Olanrewaju, O.: Enhanced intrusion detection systems performance with UNSW-NB15 data analysis. Algorithms **17**(2), 64 (2024). https://doi.org/10.3390/a17020064, https://www.mdpi.com/1999-4893/17/2/64
2. Aleksander, I., Thomas, W.V., Bowden, P.A.: WISARD: a radical step forward in image recognition. Sensor Review **4**(3), 120–124 (1984). https://doi.org/10.1108/eb007637, https://doi.org/10.1108/eb007637
3. Andronic, M., Sherborne, T., Sherborne, L., Sherborne, J., Sherborne, D., Fraser, N.J.: PolyLUT: Learning piecewise polynomials for ultra-low-latency FPGA LUT-based inference. In: Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL) (2023). https://doi.org/10.1109/FPL60245.2023.00020, https://doi.org/10.1109/FPL60245.2023.00020
4. Engelen, G., Rimmer, V., Joosen, W.: Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In: Proceedings of the IEEE Security and Privacy Workshops (SPW) (2021). https://doi.org/10.1109/SPW53761.2021.00009
5. Kull, M., Silva Filho, T., Flach, P.: Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). vol. 54, pp. 623–631. PMLR (2017)
6. Moustafa, N., Slay, J.: UNSW-NB15: A comprehensive data set for network intrusion detection systems. Military Communications and Information Systems Conference (MilCIS) (2015). https://doi.org/10.1109/MilCIS.2015.7348942, https://doi.org/10.1109/MilCIS.2015.7348942
7. Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A.: CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors **23**(13) (2023). https://doi.org/10.3390/s23135941
8. Platt, J.C.: Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In: Advances in Large Margin Classifiers. pp. 61–74. MIT Press (1999)

9. Santiago, L.A.Q., Verona, L.T., Rangel, F.d.O., Firmino, F.M., Lima, P.M.V., França, F.M.G.: FPGA implementation of weightless neural networks for network intrusion detection. In: Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN) (2020). https://doi.org/10.1109/IJCNN48605.2020.9207591, https://doi.org/10.1109/IJCNN48605.2020.9207591
10. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP) (2018). https://doi.org/10.5220/0006639801080116, https://doi.org/10.5220/0006639801080116
11. Susskind, Z., Arora, A., Bacellar, A.T.L., Dutra, D.L.C., Miranda, I.D.S., Breternitz Jr., M., Lima, P.M.V., França, F.M.G., John, L.K.: FWIW: Weightless neural network inference at ultra-low cost. In: Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA) (2023). https://doi.org/10.1145/3543622.3573140, https://doi.org/10.1145/3543622.3573140
12. Susskind, Z., Arora, A., John, L.K., John, E.B., Lima, P.M.V., França, F.M.G.: BTHOWeN – binary thermometer-encoded hashed optimized weightless neural networks. In: Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT) (2022). https://doi.org/10.1145/3559009.3569680, https://doi.org/10.1145/3559009.3569680
13. Teresa Bernarda Ludermir, André de Carvalho, A.P.B., de Souto, M.C.P.: Weightless neural models: A review of current and past works. In: Neural Computing Surveys 2 - ICSI Berkeley. pp. 41–61. Berkeley, USA (1999), https://www.cin.ufpe.br/~tbl/artigos/vol2_2-ncs-1999.pdf
14. Umuroglu, Y., Akhauri, Y., Fraser, N.J., Blott, M.: LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In: Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL) (2020). https://doi.org/10.1109/FPL50879.2020.00065, https://doi.org/10.1109/FPL50879.2020.00065
15. Zhao, Z., Sadok, H., Atre, N., Hoe, J.C., Kim, V.S., Gibbons, P.B.: Achieving 100Gbps intrusion prevention on a single server. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (2020), https://www.usenix.org/conference/osdi20/presentation/zhao-zhipeng
16. Zoghi, Z., Serpen, G.: Building an intrusion detection system on UNSW-NB15: Reducing the margin of error to deal with data overlap and imbalance. Concurrency and Computation: Practice and Experience (2024). https://doi.org/10.1002/cpe.8242, https://onlinelibrary.wiley.com/doi/full/10.1002/cpe.8242